ACADEMIC CENTRAL

PRACTICE EXAMINATION

UNIVERSITY · 2ND YEAR

OBJECT-ORIENTED PROGRAMMING

PAPER 1 · PRACTICE A · MARKING GUIDELINE

MARKS: 100

Marking guideline for the Academic Central University · 2nd year Object-Oriented Programming paper 1 · practice a.

Written by Academic Central for practice, following object-oriented programming modules at South African universities (e.g. University of Pretoria COS 110 Program Design: inheritance, polymorphism, operator overloading, memory management, encapsulation, lists, stacks and queues), in C++. It is not a university memorandum.

How to mark: each ✓ is one mark. The code shown is one correct answer. Accept any working solution that meets the requirements. Give marks for correct logic even with small syntax mistakes in QUESTIONS 3, 5 and 6.

QUESTION 1

1.1 Encapsulation: data is kept private and changed only through methods, e.g. a Motor's speed can only be set through setSpeed(), which checks the limits ✓✓
Inheritance: a new class reuses and extends an existing one, e.g. ServoMotor inherits from Actuator ✓✓
Polymorphism: one call works differently for different object types, e.g. actuator->move(90) moves a servo or a stepper correctly ✓✓
Abstraction: only the essential features are shown and the details are hidden, e.g. the program calls move() without knowing how the driver pulses are generated ✓✓ (8)

1.2 A class is a blueprint (type) that defines attributes and methods ✓; an object is one instance of a class in memory, with its own attribute values ✓ (2)

1.3 Static binding: the function to call is chosen at compile time from the type of the pointer or reference ✓
Dynamic binding: the function is chosen at run time from the actual type of the object ✓
The keyword virtual ✓ (3)

1.4 A class with at least one pure virtual function, e.g. virtual void move(double d) = 0; ✓. It cannot be instantiated, and derived classes must override the pure virtual functions ✓ (2)

[15]

QUESTION 2

2.1 AB|~B~A ✓✓✓✓ (base constructed first; destroyed in reverse order) (4)

2.2 lidar S ✓✓✓✓ (read() is virtual, so Lidar's version runs; id() is not, so Sensor's version runs) (4)

2.3 1 6 ✓✓✓ (3)

2.4 5 ✓✓✓ (a, b, c and the two elements of d) (3)

2.5 8,2 ✓✓✓ (3)

2.6 4 3 ✓✓✓ (3)

[20]

QUESTION 3

3.1 Actuator (abstract, name in italics or marked {abstract}): # name : string; + Actuator(n : string); + getName() : string; + move(degrees : double) {abstract} ✓✓
ServoMotor: − angle : double; + ServoMotor(n : string); + move(degrees : double); + getAngle() : double ✓
StepperMotor: − position : int; − STEPS_PER_REV : int = 200; + StepperMotor(n : string); + move(degrees : double); + getPosition() : int ✓
Hollow-headed inheritance arrows from both subclasses to Actuator ✓
Correct visibility symbols ✓ (6)

3.2 Abstract class ✓ protected name ✓ constructor ✓ pure virtual move ✓ virtual destructor ✓ (5)

```
class Actuator {
protected:
    string name;
public:
    Actuator(const string& n) : name(n) {}
    string getName() const { return name; }
    virtual void move(double degrees) = 0;
    virtual ~Actuator() {}
};
```

3.3 Inherits publicly ✓ calls the base constructor ✓ private angle starting at 0 ✓ override of move ✓ clamping to 0–180 ✓✓ (6)

```
class ServoMotor : public Actuator {
    double angle;
public:
    ServoMotor(const string& n) : Actuator(n), angle(0) {}
    void move(double degrees) override {
        angle += degrees;
        if (angle < 0) angle = 0;
        if (angle > 180) angle = 180;
    }
    double getAngle() const { return angle; }
};
```

3.4 Inherits and constructs correctly ✓ position attribute ✓ conversion degrees × 200 / 360 ✓ rounding ✓ added to position ✓ (5)

```
class StepperMotor : public Actuator {
    int position;
    static const int STEPS_PER_REV = 200;
public:
    StepperMotor(const string& n) : Actuator(n), position(0) {}
    void move(double degrees) override {
        position += (int) round(degrees * STEPS_PER_REV / 360.0);
    }
    int getPosition() const { return position; }
};
```

3.5 Vector of base pointers with both objects ✓ polymorphic loop ✓ delete each object ✓ (3)

```
int main() {
    vector<Actuator*> parts;
    parts.push_back(new ServoMotor("wrist"));
    parts.push_back(new StepperMotor("base"));
    for (Actuator* a : parts) a->move(90);
    for (Actuator* a : parts) delete a;
}
```

[25]

QUESTION 4

4.1 Stack: automatic memory for local variables, freed when the function returns ✓ (2)
Heap: memory requested at run time with new; it stays until it is freed with delete ✓

4.2 Line A: returns the address of a local array that is destroyed when the function ends (dangling pointer) ✓; fix: allocate with new int[10] (or return a vector) ✓ (6)
Line B: the first array of 5 is lost and can never be freed (memory leak) ✓; fix: delete[] p before giving it a new array ✓
Line C: q points to the same memory as p, so it is deleted twice (double delete, undefined behaviour) ✓; fix: delete it only once and set the pointers to nullptr ✓

4.3 Shallow copy: only the pointer is copied, so two objects share the same array ✓ (4)
Deep copy: a new array is allocated and the values are copied ✓
Without a deep copy constructor and assignment operator, changing one object changes the other ✓, and both destructors delete the same memory (double delete); the destructor is needed to free the array ✓

4.4 unique_ptr owns the object and deletes it automatically when it goes out of scope ✓, even when an exception occurs ✓; it cannot be copied, so the object has exactly one owner and cannot be deleted twice ✓ (3)

[15]

QUESTION 5

5.1 struct with data and next ✓ new node ✓ next set to the old head ✓ head updated ✓ head passed by reference ✓ (5)

```
struct Node {
    int data;
    Node* next;
};
void pushFront(Node*& head, int value) {
    Node* n = new Node{value, head};
    head = n;
}
```

5.2 Stack: last in, first out (LIFO) ✓; e.g. storing the path so the robot can retrace its steps back to the start, or undo commands ✓ (4)
Queue: first in, first out (FIFO) ✓; e.g. a list of commands or waypoints carried out in the order they arrive, or incoming serial messages ✓

5.3 Fixed array of 5 ✓ index and count attributes ✓ add() writes at the index ✓ index wraps around with % 5 ✓ average() over count values ✓ no division by zero ✓ (6)

```
class SensorBuffer {
    double data[5];
    int next = 0, count = 0;
public:
    void add(double r) {
        data[next] = r;
        next = (next + 1) % 5;
        if (count < 5) count++;
    }
    double average() const {
        if (count == 0) return 0;
        double sum = 0;
        for (int i = 0; i < count; i++) sum += data[i];
        return sum / count;
    }
};
```

[15]

QUESTION 6

6.1 template line ✓ parameters of type T ✓ correct return ✓ (3)

```
template <typename T>
T largest(T a, T b) {
    return (a > b) ? a : b;
}
```

6.2 Condition ✓ throw out_of_range with a message ✓ try block ✓ catch by reference and print what() ✓ (4)

```
void setSpeed(int s) {
    if (s < 0 || s > 100) throw out_of_range("Speed must be 0 to 100");
    // set the speed here
}
int main() {
    try {
        setSpeed(150);
    } catch (const out_of_range& e) {
        cout << e.what() << endl;
    }
}
```

6.3 The compiler then stops the function from changing the object by mistake ✓, and the function can be called on const objects and const references ✓ (3)
Example: double getAngle() const { return angle; } ✓

[10]

TOTAL: 100

END OF MARKING GUIDELINE